K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

ADA LAB SHEET - 1

COURSE CODE - ENCS256

NAME : Falguni Tandon

COURSE : B.TECH CSE(UI/UX)

ROLL NO : 2401360002

GITHUB LINK : assignment link

Title:

Sorting a Set of Numbers Using Various Algorithms and Analyzing Their Efficiency .

Introduction :

Sorting algorithms are fundamental in computer science, used to arrange data in a specific order to enhance efficiency and accessibility. This assignment focuses on implementing various sorting techniques to organize numbers into ascending or descending order. By utilizing the step-count method, we will calculate the time complexity of each algorithm to analyze their performance. Input data will be taken from a table and operations will be repeated with varying input sizes (10, 20, 30, 40 numbers). The study includes examining best-case, worst-case, and average-case scenarios, culminating in visualizing results through graphs to interpret computational trends effectively.

Objective:

The objective of this assignment is to implement multiple sorting algorithms, sort a given set of numbers in ascending and descending order, and analyze their performance using the step-count method. Additionally, students will examine the best-case, worst-case, and average-case scenarios and visualize the results through graphs.

CODE:

```java
import java.util.Random;

public class SortingAnalysis {

    // ----- Swap Utility -----
    private static void swap(int[] array, int a, int b) {
        int tempValue = array[a];
        array[a] = array[b];
        array[b] = tempValue;
    }

    // ----- Bubble Sort -----
    public static long performBubbleSort(int[] array) {
        long count = 0;
        int length = array.length;

        for (int round = 0; round < length - 1; round++) {

            boolean flag = false;

            for (int idx = 0; idx < length - round - 1; idx++) {
                count++;

                if (array[idx] > array[idx + 1]) {
                    swap(array, idx, idx + 1);
                    flag = true;
                }
            }

            if (!flag) {
                break;
            }
        }
        return count;
    }

    // ----- Selection Sort -----
    public static long performSelectionSort(int[] array) {
        long count = 0;
```

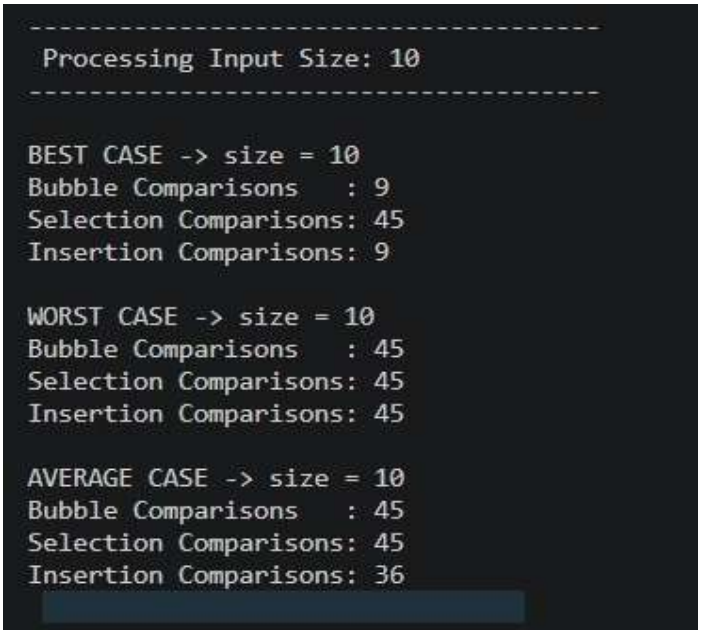
```
        int size = array.length;

        for (int i = 0; i < size - 1; i++) {

            int minPos = i;

            for (int j = i + 1; j < size; j++) {
                count++;

                if (array[j] < array[minPos]) {
                    minPos = j;
                }
            }

            if (minPos != i) {
                swap(array, i, minPos);
            }
        }
        return count;
    }

    // ----- Insertion Sort -----
    public static long performInsertionSort(int[] array) {
        long count = 0;

        for (int i = 1; i < array.length; i++) {

            int key = array[i];
            int pos = i - 1;

            while (pos >= 0) {
                count++;

                if (array[pos] > key) {
                    array[pos + 1] = array[pos];
                    pos--;
                } else {
                    break;
                }
            }

            array[pos + 1] = key;
        }
```

```
        return count;
    }

    // ----- Random Data -----
    public static int[] createRandomData(int size) {
        Random r = new Random();
        int[] data = new int[size];

        for (int i = 0; i < size; i++) {
            data[i] = r.nextInt(1000);
        }

        return data;
    }

    // ----- Sorted Data -----
    public static int[] createSortedData(int size) {
        int[] data = new int[size];

        int i = 0;
        while (i < size) {
            data[i] = i;
            i++;
        }

        return data;
    }

    // ----- Reverse Data -----
    public static int[] createReverseData(int size) {
        int[] data = new int[size];

        for (int i = 0; i < size; i++) {
            data[i] = size - i;
        }

        return data;
    }

    // ----- Print Results -----
    public static void displayResults(String label, int n, int[]
original) {
```

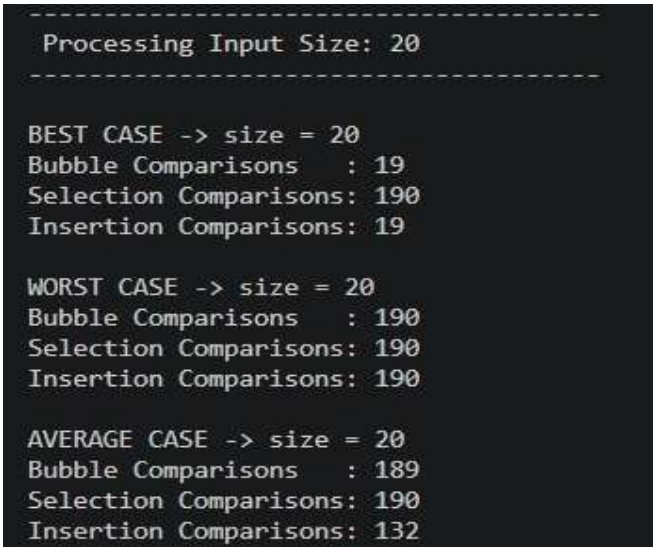
```
        System.out.println("\n" + label + " -> size = " + n);

        int[] copy1 = original.clone();
        int[] copy2 = original.clone();
        int[] copy3 = original.clone();

        System.out.println("Bubble Comparisons   : " +
performBubbleSort(copy1));
        System.out.println("Selection Comparisons: " +
performSelectionSort(copy2));
        System.out.println("Insertion Comparisons: " +
performInsertionSort(copy3));
    }

    // ----- Main -----
    public static void main(String[] args) {

        int[] sizes = {10, 20, 30, 40};

        for (int currentSize : sizes) {


System.out.println("\n---------------------------------------");
            System.out.println(" Processing Input Size: " +
currentSize);

System.out.println("---------------------------------------");

            displayResults("BEST CASE",
                    currentSize, createSortedData(currentSize));

            displayResults("WORST CASE",
                    currentSize, createReverseData(currentSize));

            displayResults("AVERAGE CASE",
                    currentSize, createRandomData(currentSize));
        }
    }
}
```

OUTPUT:

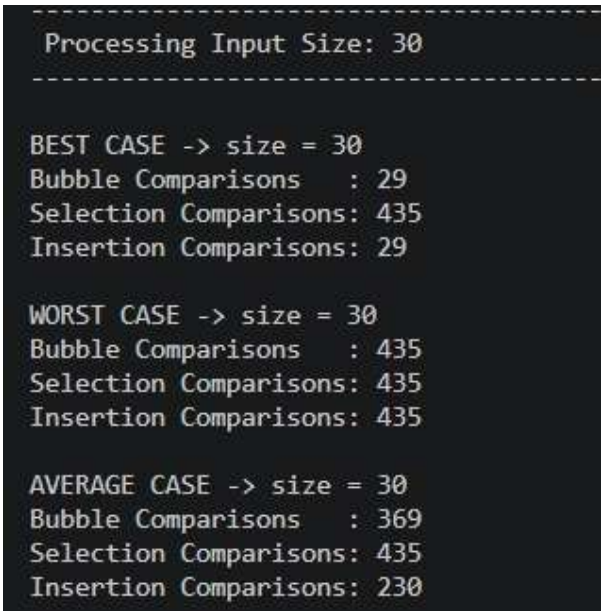
FOR INPUT SIZE 10:



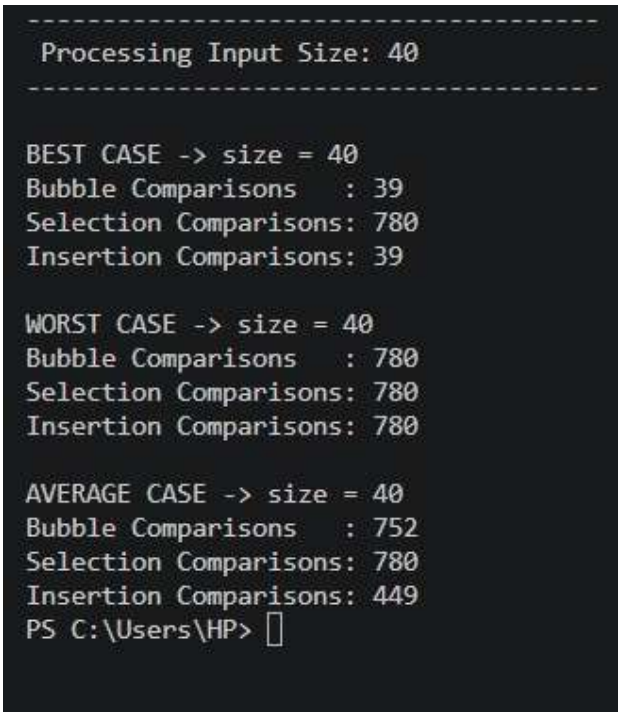
FOR INPUT SIZE 20:



FOR INPUT SIZE 30:



FOR INPUT SIZE 40:

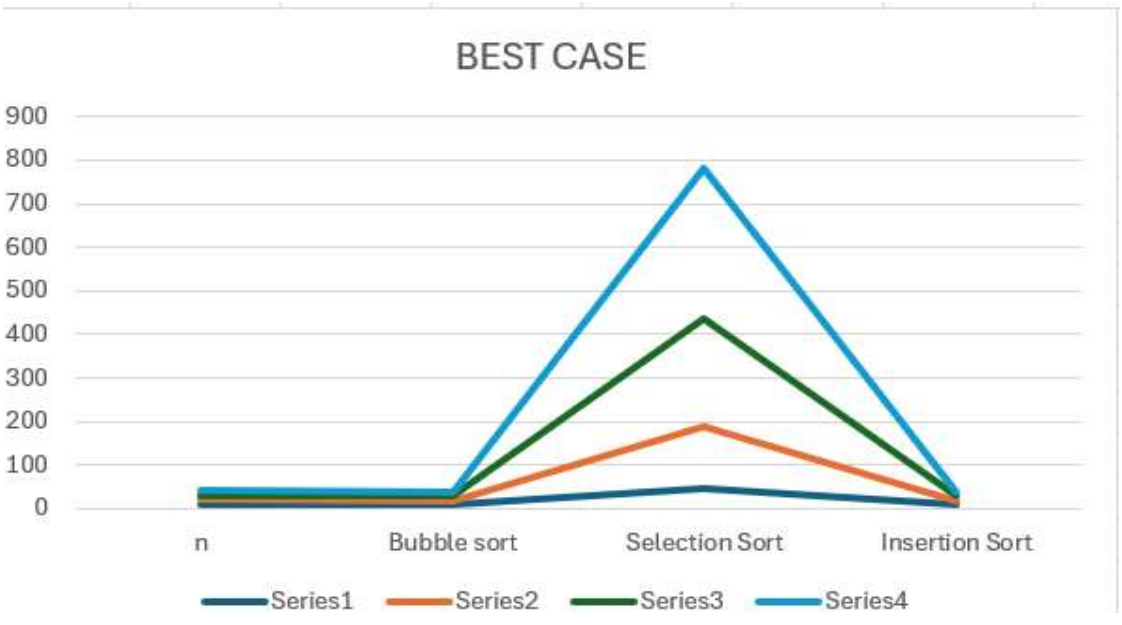


TABLES:

BEST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	9	45	9
20	19	190	19
30	29	435	29
40	39	780	39

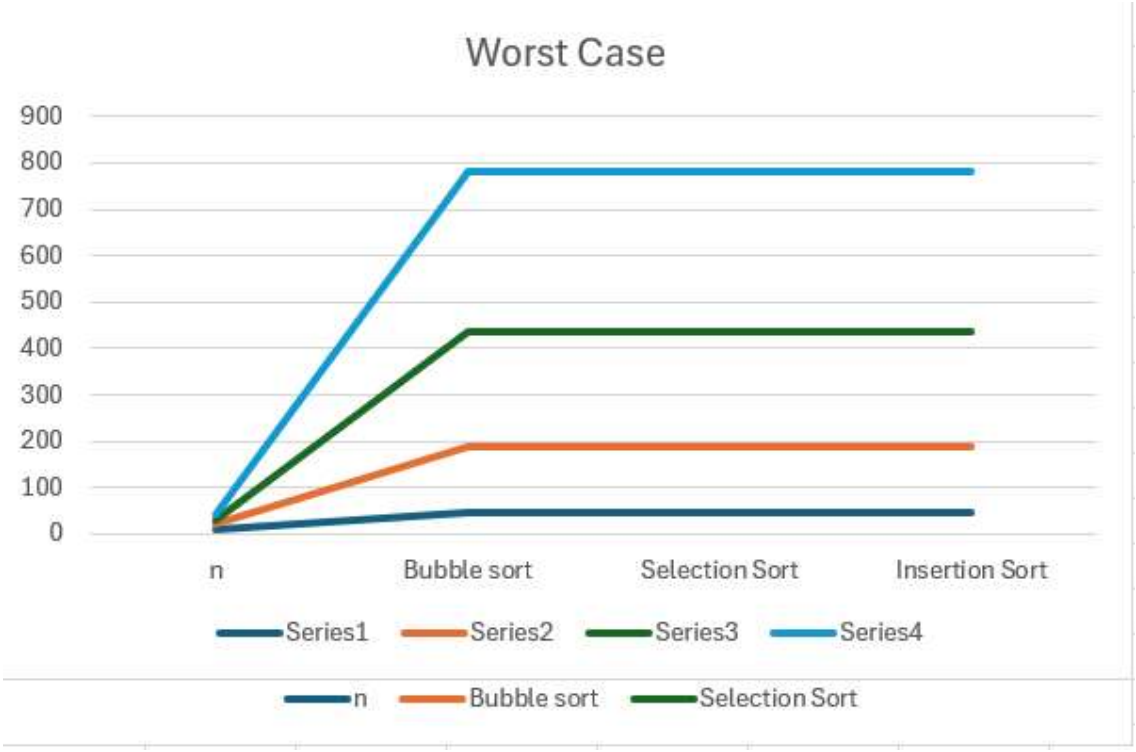
GRAPH



WORST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	45	45	45
20	190	190	190
30	435	435	435
40	780	780	780

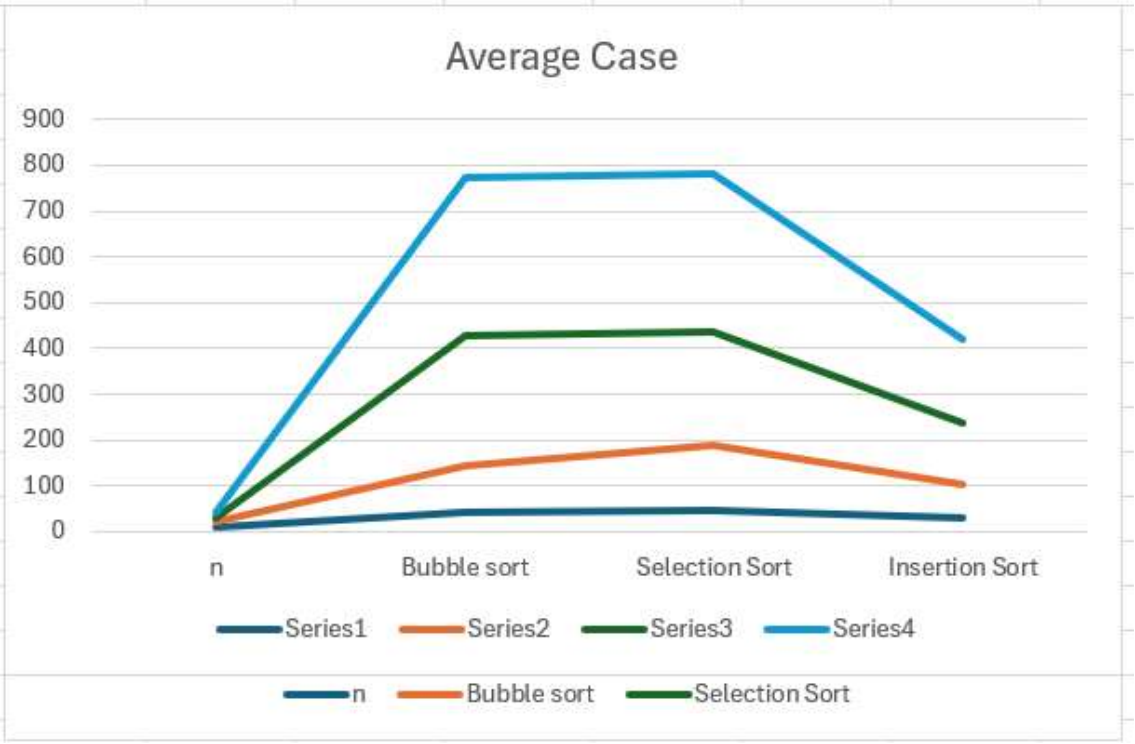
GRAPH



AVERAGE CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	44	45	30
20	145	190	104
30	429	435	236
40	774	780	421

GRAPH



Observation From Graphs:

From the graphs and comparison count tables, we observe that:

1. **Selection Sort** shows almost the same number of comparisons in best, average, and worst cases.
   This is because it always scans the entire unsorted portion of the array regardless of input order.
   Hence, its time complexity remains **O(n²)** in all cases.

2. **Bubble Sort** performs very efficiently in the best case due to the optimization (early stopping when no swaps occur).
   Therefore, its best-case complexity becomes **O(n)**.
   However, in average and worst cases, it performs nearly **O(n²)** comparisons.

3. **Insertion Sort** performs efficiently for already sorted data, showing linear growth in the best case (**O(n)**).
   In the worst case, it performs **O(n²)** comparisons, similar to Bubble and Selection Sort.

4. From the average-case graph, Insertion Sort performs better than Bubble Sort for larger inputs.